

Chapter 8: I/O modules:

CHAPTER 8 — INPUT/OUTPUT

BATCH 1 — Why We Need I/O Modules + External Devices + I/O Module Functions

This batch covers textbook pages **1–10**.

Citations included:

Chapter 8 - Input_Output

1. WHY CAN'T WE CONNECT PERIPHERALS DIRECTLY TO THE CPU OR SYSTEM BUS?

Chapter 8 - Input_Output

According to page 2 of the chapter, this is **not just impractical — it's impossible** for several reasons:

A. Peripherals differ wildly in behavior

- Keyboards → human-speed
- SSDs → extremely fast
- Printers → mechanical
- Sensors → real-time signals

The CPU would require **custom logic for every device**. Absurd.

B. Peripherals run at totally different speeds

- CPU/memory operate at **nanoseconds**
- Many devices operate at **microseconds or milliseconds**

This mismatch means:

- Direct CPU–device transfers waste CPU cycles because CPU must wait.
- Or the device overwhelms CPU because it bursts too fast for memory.

C. Data formats differ

Devices may use:

- 7-bit ASCII
- Variable-width sensor packets

- Sector-based disk blocks
The CPU expects machine-word width (32, 64 bits).
⇒ Need translation.

D. System bus cannot be tied up by slow devices

If a keyboard (1 byte per key press) hogged a 64-bit high-speed system bus → catastrophic inefficiency.

Conclusion:

The CPU needs an agent to sit between it and the messy real world.

That agent = **I/O Module**.

2. GENERIC MODEL OF AN I/O MODULE

(See page 3, Fig. 7.1)

Chapter 8 - Input_Output

An I/O module sits between:

- **System bus**
- **Peripheral device(s)**

It has:

- **Data lines** ↔ system bus
- **Control lines** (status, commands)
- **Address lines** (which device?)
- **Links to external devices**

Core idea:

CPU speaks "bus protocol".

Devices speak "their own weird protocols".

I/O module is the **translator**.

3. EXTERNAL DEVICES: HUMAN-READABLE, MACHINE-READABLE, COMMUNICATION

(p.4)

Chapter 8 - Input_Output

External devices fall into 3 categories:

A. Human-readable

Used by humans:

- Monitors
- Printers
- Keyboards

B. Machine-readable

Used by equipment:

- Hard disks
- Tapes
- Sensors
- Actuators

C. Communication devices

Used to talk to remote systems:

- Network cards
- Modems
- Routers

Each type has different timing, protocols, formats → reinforcing why I/O modules are essential.

4. BLOCK DIAGRAM OF AN EXTERNAL DEVICE

(page 5, Fig. 7.2)

Chapter 8 - Input_Output

Every external device contains:

Transducer

- Converts external physical signals → digital data or vice-versa.

Examples:

- Keyboard: mechanical movement → scan code
- Microphone: sound waves → electric signal
- Disk head: magnetic regions → bits

Buffer

Device's temporary data storage.

Why? Because devices usually operate **slower or faster** than the I/O module.

Control logic

Handles:

- READY/BUSY states
- Errors (paper jam, disk fault)
- Interpretation of control signals from I/O module

Status + Control Signals

These lines tell CPU:

- READY?
- BUSY?
- ERROR?

5. FUNCTIONS OF AN I/O MODULE

(page 6)

Chapter 8 - Input_Output

There are *five fundamental responsibilities*:

A. Control and Timing

Coordinates:

- When CPU wants data
- When device can send data

- When device is ready
- When CPU can receive

I/O module sequences multi-step interactions:

- Check device status
 - Command device
 - Initiate transfer
 - Respond to interrupts
-

B. Processor Communication

I/O module must support:

1. Command decoding

E.g. for a disk module:

- READ SECTOR
- WRITE SECTOR
- SEEK
- FORMAT

2. Data transfer

Bidirectional over bus.

3. Status reporting

Common signals:

- **READY**
- **BUSY**
- **ERROR**

CPU uses this to know when to proceed.

4. Address recognition

Each I/O device has a unique address.

C. Device Communication

I/O module speaks device-specific protocol:

- Device commands
- Data packaging
- Error signals
- Timing signaling

CPU NEVER speaks directly to the device.

D. Data Buffering

(p.9)

Chapter 8 - Input_Output

Why buffer?

CPU/memory operate in **nanoseconds**.

Devices often operate in **micro/milliseconds**.

Example:

- CPU → 64 bytes in 20 ns
- Printer → prints one character per ms

If CPU waited for printer: system collapse.

Solution:

I/O module buffers data:

- CPU gives data in fast bursts
 - Device consumes slowly (or vice versa)
-

E. Error Detection

Covers:

- Mechanical failures (bad disk track)
- Electrical problems
- Transmission errors (bit flips)

Typical mechanisms:

- **Parity bits** (7-bit ASCII + parity → 8th bit)
- **CRC** for block transfers

I/O module reports errors upwards to CPU.

6. PROCESSOR-I/O CONTROL FLOW EXAMPLE

(p.7)

Chapter 8 - Input_Output

A sample sequence (simplified):

1. CPU checks status → READY?
2. Device says READY
3. CPU sends READ command
4. Device retrieves one unit of data
5. CPU reads data
6. All interactions must arbitrated through system bus

Important point:

CPU must check status repeatedly in programmed I/O (wasteful).

CHAPTER 8 — BATCH 2

PROGRAMMED I/O • INTERRUPT-DRIVEN I/O • INTERRUPT PROCESSING • INTERRUPT IDENTIFICATION

(Covering textbook pages **10–22**)

Citations reference *Chapter 8 – Input_Output.pdf*.

=====

1. PROGRAMMED I/O (the dumbest, slowest method)

=====

(Page 10–12)

Chapter 8 - Input_Output

Programmed I/O = CPU does **everything**.

Sequence for a programmed I/O read:

1. CPU checks I/O module status bit: READY?
2. If not ready → CPU loops (busy-wait).
3. Once ready → CPU issues READ command.
4. CPU waits again.
5. I/O module transfers data into its buffer.
6. CPU reads data from the I/O module.

The fatal flaw:

The CPU is **blocked**, doing nothing but polling a device.

Example:

Keyboard ready once per **~100 ms** → CPU wastes millions of cycles spinning.

This method is listed in the textbook as:

- **Simple,**
- **No hardware complexity,**
- **Disastrous CPU utilization.**

Used only when:

- Low-speed devices,
- Simplicity matters more than performance.

2. INTERRUPT-DRIVEN I/O

(Pages 12–15)

Chapter 8 - Input_Output

Interrupt-driven I/O fixes Programmed I/O's stupidity.

Basic idea:

CPU does *real work*, I/O module signals the CPU **when it needs attention**.

Sequence:

1. CPU issues I/O command and **continues execution**.
2. Device becomes READY.

3. I/O module sends **interrupt** to CPU.
4. CPU stops current execution → saves context.
5. CPU services interrupt:
 - Reads/Writes data
 - Updates buffers
6. CPU restores context → continues previous work.

Benefits:

- CPU no longer wastes cycles waiting.
- Good for moderate-speed devices (keyboard, serial, USB).

Still not perfect:

- CPU still moves data between I/O module and memory → expensive for large blocks.

This leads to **DMA**, covered in Batch 3.

=====

3. INTERRUPT CYCLE (VERY IMPORTANT EXAM DIAGRAM)

=====

(Page 14–15)

Chapter 8 - Input_Output

CPU executes instructions in fetch–execute cycle.

During the **interrupt check stage**, CPU reads interrupt lines.

If interrupt line = HIGH → CPU:

1. Finishes current instruction.
2. Saves PC, registers, PSW.
3. Loads interrupt vector.
4. Jumps to interrupt service routine (ISR).

Classic exam trick:

Instruction execution is **not** interrupted mid-instruction.

Interrupt is latched → serviced *between* instructions.

=====

4. INTERRUPT PROCESSING STEPS

=====

(Page 15–17)

Chapter 8 - Input_Output

A. Save processor state

Program counter + registers saved to stack or special registers.

B. Identify the source of interrupt

More on this in next subsection.

C. Transfer control to ISR

Interrupt vector table holds ISR addresses.

D. Execute ISR

Usually:

- Read device data
- Clear interrupt flag
- Possibly wake a blocked process

E. Restore state

Return-from-interrupt instruction (RTI).

=====

5. INTERRUPT IDENTIFICATION

(Polling + Daisy-Chain + Bus Arbitration + Vectored Interrupts)

=====

(Page 17–19)

Chapter 8 - Input_Output

This section is extremely exam-heavy.

Method 1 — Multiple Interrupt Lines

Each device has its own interrupt pin.

→ Fast but expensive (lots of wires).

Method 2 — Software Polling

CPU asks each device:

- “Did YOU cause the interrupt?”

Terrible performance → $O(N)$ scanning.

Method 3 — Daisy-Chain (Hardware Priority)

(Page 18)

One interrupt line goes through many devices, chained:

CPU ← Device 1 ← Device 2 ← Device 3 ← ...

- Closest device has highest priority.
 - Each device can “block” the chain if it needs service.
-

Method 4 — Bus Arbitration

Devices compete for bus control → bus controller picks winner.

More flexible than daisy chain.

Method 5 — Vectored Interrupts (BEST METHOD)

(Page 19)

Device that signals interrupt sends an **interrupt vector** (address of ISR) onto the bus.

CPU jumps directly to the correct ISR WITHOUT polling.

Almost all real architectures use vectored interrupts:

- x86
 - ARM
 - RISC-V
 - MIPS
-

=====

6. 8259A PROGRAMMABLE INTERRUPT CONTROLLER

=====

(Page 20–22)

Chapter 8 - Input_Output

You must know the conceptual role, not the exact pins.

Purpose:

- Manages **up to 8 interrupt lines**
- Can be cascaded for up to **64 devices**
- Prioritizes requests
- Sends vectored interrupts to CPU

Functions:

- **Interrupt Request Register (IRR)** — holds pending interrupts
- **In-Service Register (ISR)** — marks interrupt being serviced
- **Interrupt Mask Register (IMR)** — masks interrupts
- **Priority resolver**

CPU interacts with 8259A instead of directly handling 100 devices.

END OF BATCH 2

The heavy theory is done.

Now comes DMA and advanced I/O architecture.

CHAPTER 8 — BATCH 3

DIRECT MEMORY ACCESS (DMA) • DMA CYCLE STEALING • BUS ARBITRATION • I/O CHANNELS & PROCESSORS

(Covers textbook pages **22–29**)

=====

1. Why DMA Exists

=====

(Page 22)

Chapter 8 - Input_Output

Interrupt-driven I/O still requires CPU to **move every byte** between I/O module and memory.

This is insane for large transfers (e.g., disk blocks, network packets).

Example:

Reading 4 KB from disk = CPU handles 4096 byte transfers.

Solution:

CPU delegates the job to **DMA controller**.

=====

2. DMA CONCEPT

=====

DMA = Direct Memory Access

A **DMA module** transfers blocks of data directly between memory and I/O devices **without CPU intervention**.

CPU only does:

1. Set starting memory address
2. Set word count
3. Set direction (read/write)
4. Start DMA

Then DMA takes over.

=====

3. DMA OPERATION MODES

=====

(Page 24)

Chapter 8 - Input_Output

A. DMA Steals Bus Cycles (“Cycle Stealing”)

During CPU execution, DMA temporarily takes control of memory bus to perform a single transfer.

CPU slows down slightly because some cycles are stolen.

B. Block Transfer Mode (“Burst Mode”)

DMA takes full bus control and transfers entire block at once.

CPU is frozen during transfer.

Faster than cycle stealing but more disruptive.

C. Transparent DMA

DMA only transfers when CPU is not using the bus.

Least disruptive but slowest.

=====

4. DMA CYCLE (step-by-step)

=====

(Page 23)

Chapter 8 - Input_Output

1. CPU sets up DMA registers (address, count, control).
2. DMA requests bus from CPU.
3. CPU acknowledges.
4. DMA transfers one word to/from memory.
5. DMA updates address and count.
6. DMA repeats until count = 0.
7. DMA sends interrupt to CPU signaling completion.

=====

5. DMA vs INTERRUPT I/O vs PROGRAMMED I/O

=====

Method	Who moves data?	CPU involvement	Speed
Programmed	CPU	Constant polling	Slowest
Interrupt	CPU	Per transfer	Medium

Method	Who moves data?	CPU involvement	Speed
DMA	DMA controller	CPU sets up only	Fastest

6. I/O CHANNELS AND I/O PROCESSORS

(Page 28–29)

Chapter 8 - Input_Output

DMA handles raw transfers, but complex systems need more.

I/O Channel

- A programmable DMA engine.
- Executes I/O-specific instructions.
- Used in mainframes.

I/O Processor

- A separate processor dedicated entirely to I/O.
- Can manage multiple DMA controllers.
- Offloads all I/O burdens from CPU.

Modern analogy:

- NVMe SSD controllers
- NIC offload engines
- GPU copy engines

END OF BATCH 3

Now we finish with the last batch:

I/O Mapping, Buses, PCI, PCIe, FireWire, USB.

CHAPTER 8 — BATCH 4

I/O MAPPING • BUS ARCHITECTURE • PCI • PCI EXPRESS • USB

(Covers textbook pages **29–38**)

=====

1. I/O MAPPING — MEMORY-MAPPED I/O vs ISOLATED I/O

=====

(Page 29–30)

Chapter 8 - Input_Output

A. Memory-Mapped I/O

I/O registers share same address space as memory.

Advantages:

- CPU uses normal load/store instructions
- Flexible
- Simpler instruction set

Disadvantage:

- Memory address space is reduced (part reserved for I/O)

Used in ARM, RISC-V, MIPS.

B. Isolated I/O (I/O-mapped I/O)

I/O uses separate address space.

CPU has special instructions:

- IN
- OUT

Used in x86 architecture.

=====

2. BUS ARCHITECTURE OVERVIEW

=====

(Page 31–32)

Types of buses:

- **System bus** (CPU ↔ memory)
- **I/O bus** (slower peripherals)
- **Backplane bus** in servers
- **High-speed point-to-point interconnects** (PCIe)

Bus Control:

- Synchronous
- Asynchronous

Bus Arbitration:

Methods include:

- Daisy chain
- Centralized controller
- Distributed arbitration

=====

3. PCI BUS

=====

(Page 33–34)

PCI = **P**eripheral **C**omponent **I**nterconnect

Features:

- 32-bit or 64-bit bus
- Up to 133 MB/s
- Burst mode
- Bus mastering (device controls bus)

Bus mastering is important:

A PCI device may become a bus master and transfer data without CPU intervention → similar to DMA.

4. PCI EXPRESS (PCIe) — Modern Standard

(Page 34–36)

Chapter 8 - Input_Output

PCIe replaces parallel bus with **high-speed serial lanes**.

Key benefits:

- Point-to-point links
- No bus contention
- Multi-lane scaling (x1, x4, x8, x16)
- Gigabytes per second bandwidth
- Used for GPUs, SSDs, NICs

Signaling:

- Packet-based (like network protocol)
- Full-duplex

PCIe is one of the most important interfaces in modern computing.

5. USB — UNIVERSAL SERIAL BUS

(Page 37–38)

Chapter 8 - Input_Output

USB changed everything.

Goals:

- Replace dozens of incompatible connectors
- Support plug-and-play
- Provide power and data
- Hot-swappable devices

USB Features:

- **Tiered star topology**
- **Host controller controls everything**
- **Four transfer types:**
 - Control
 - Bulk
 - Interrupt
 - Isochronous (for audio/video)

USB Speeds:

- USB 1.1 → 12 Mbps
- USB 2.0 → 480 Mbps
- USB 3.x → 5–20 Gbps

Real-world insight:

USB uses **polling from host**, not true hardware interrupts from devices.